Tutoria_88_14_PUCPR_DataSet_STJ_[Decisões]

November 25, 2025

#Dataset Dados Abertos STJ - Íntegras de Decisões Terminativas e Acórdãos do Diário da Justiça (Decisões)

##Experimento 14: Max Pooling

##Importação de bibliotecas e declaração de variáveis globais

[1]:
```
ENVIRONMENT = 'PUCPR'

import torch

# Check if a GPU (CUDA) is available
if torch.cuda.is_available():
    print("GPU is available!")
    print("Device Name:", torch.cuda.get_device_name(0))
else:
    print("GPU not available, using CPU.")

# This is the line you'd use in your script to send your model or data to the␣
 ↪device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
GPU is available!
Device Name: Tesla V100S-PCIE-32GB
Using device: cuda
```

##Parametrização conforme ambiente de execução.

[2]:
```
import pandas as pd

# Define the base path to the dataset folder
if ENVIRONMENT == 'COLAB':
  from google.colab import drive
  drive.mount('/content/drive')

  base_path = '/content/drive/MyDrive/_Mestrado PUCPR/Datasets/'
  num_layers_to_freeze = 165 # You can adjust this number based on your␣
 ↪inspection of layer names
```

```
  #model_checkpoint = 'neuralmind/bert-base-portuguese-cased'
  model_checkpoint = 'neuralmind/bert-large-portuguese-cased'
  EPOCHS = 3
  APPROACH_EXECUTION = 'step'
  reduced_embedding_dim = 512
elif ENVIRONMENT == 'PUCPR':
  num_layers_to_freeze = 165 # You can adjust this number based on your␣
 ↪inspection of layer names
  base_path = 'dataset/'
  model_checkpoint = 'neuralmind/bert-large-portuguese-cased'
  EPOCHS = 100
  APPROACH_EXECUTION = 'epoch'
  reduced_embedding_dim = 1024
else:
  raise Exception("Environment of execution is not recognize!")

install_evaluate = False
```

##Carregar classe/rótulo para a decisão (Favorável ou Não ao Réu)

[3]:
```
# Define file paths
file_path = base_path + 'descricao-decisao-julgamento.csv'

# Load each CSV file into a dataframe
try:
    dfTipoDecisao = pd.read_csv(file_path, sep=';')
    print(f"Successfully loaded {file_path} into dataframe 'dfTipoDecisao'")
    display(dfTipoDecisao.head())
except FileNotFoundError:
    print(f"Error: File not found at {file_path}")
except Exception as e:
    print(f"An error occurred while loading {file_path}: {e}")
```

```
Successfully loaded dataset/descricao-decisao-julgamento.csv into dataframe
'dfTipoDecisao'
```

	COD_TEOR_DECISAO	DESC_TEOR_DECISAO	DESC_RSULT_JGTO
0	1	Concedendo	AGRAVO CONHECIDO EM PARTE E, NESSA PARTE, PROVIDO
1	1	Concedendo	Agravo regimental provido
2	1	Concedendo	CONCEDENDO HABEAS CORPUS
3	1	Concedendo	CONCEDENDO HABEAS DATA
4	1	Concedendo	CONCEDENDO MANDADO DE SEGURANÇA

```
                  RESULTADO_RESUMIDO DECISAO_FAVORAVEL
0  PARCIALMENTE CONHECIDO E PROVIDO                  S
1                           PROVIDO                  S
2                         CONCEDIDO                  S
3                         CONCEDIDO                  S
4                         CONCEDIDA                  S
```

##Carregar metadados das decisões (Documento, Processo, Nro de Registro, Ministro, Teor).

[4]:
```
file_path = base_path + 'metadados-decisoes-acordaos-dj052025.csv'

try:
    dfMetaDadosDecisao = pd.read_csv(file_path, sep=';')
    print(f"Successfully loaded {file_path} into dataframe␣
 ↪'dfMetaDadosDecisao'")
    display(dfMetaDadosDecisao.head(3))
except FileNotFoundError:
    print(f"Error: File not found at {file_path}")
except Exception as e:
    print(f"An error occurred while loading {file_path}: {e}")
```

```
Successfully loaded dataset/metadados-decisoes-acordaos-dj052025.csv into
dataframe 'dfMetaDadosDecisao'
```

```
   SEQDOCUMENTO DATAPUBLICACAO TIPODOCUMENTO  NUMEROREGISTRO       PROCESSO  \
0     309631639     2025-05-05       DECISÃO    202000946481        TP 2695
1     310378377     2025-05-05       DECISÃO    202400362531  AREsp 2562174
2     309409286     2025-05-05       DECISÃO    202401080738   REsp 2132967

  DATARECEBIMENTO DATADISTRIBUICAO             NM_MINISTRO RECURSO  \
0      2020-04-25       2020-04-27   MARIA ISABEL GALLOTTI     NaN
1      2024-02-09       2024-02-28  JOÃO OTÁVIO DE NORONHA     NaN
2      2024-04-02       2024-04-04      MESSOD AZULAY NETO     NaN

         TEOR                                DESCRICAOMONOCRATICA  \
0     Negando  Prejudicada a ação de #{nome_da_parte} #{campo…
1     Negando  Conhecido o recurso de #{nome_da_parte} e não-…
2  Concedendo  Conhecido o recurso de #{nome_da_parte} e prov…

                  ASSUNTOS
0                     4993
1  11806, 7752, 7780, 7779
2               3416, 3435
```

##Carregar detalhes das decisões

[5]:
```
# Define file paths
file_path = base_path + 'decisoes-acordaos-dj052025.csv'
```

```
try:
    dfDetalheDecisao = pd.read_csv(file_path, sep=';')
    print(f"Successfully loaded {file_path} into dataframe 'dfDetalheDecisao'")
    display(dfDetalheDecisao.head(3))
except FileNotFoundError:
    print(f"Error: File not found at {file_path}")
except Exception as e:
    print(f"An error occurred while loading {file_path}: {e}")
```

```
Successfully loaded dataset/decisoes-acordaos-dj052025.csv into dataframe
'dfDetalheDecisao'
```

	SEQDOCUMENTO	ACORDAO	EMENTA	RELATORIO	VOTO	DECISAO
0	309631639	NaN	NaN	NaN	NaN	O presente pedido de contracautela tinha por o…
1	310378377	NaN	NaN	NaN	NaN	Trata-se de agravo em recurso especial interpo…
2	309409286	NaN	NaN	NaN	NaN	Trata-se de recurso especial interposto por MA…

##Tratamento dos dados ###Exclusão de campos desnecessários

[6]:
```
dfMetaDadosDecisao.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48588 entries, 0 to 48587
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   SEQDOCUMENTO          48588 non-null  int64
 1   DATAPUBLICACAO        48588 non-null  object
 2   TIPODOCUMENTO         48588 non-null  object
 3   NUMEROREGISTRO        48588 non-null  int64
 4   PROCESSO              48588 non-null  object
 5   DATARECEBIMENTO       48588 non-null  object
 6   DATADISTRIBUICAO      48588 non-null  object
 7   NM_MINISTRO           48588 non-null  object
 8   RECURSO               6200 non-null   object
 9   TEOR                  48588 non-null  object
 10  DESCRICAOMONOCRATICA  48588 non-null  object
 11  ASSUNTOS              48588 non-null  object
dtypes: int64(2), object(10)
memory usage: 4.4+ MB
```

[7]:
```
dfMetaDadosDecisao.head(3)
```

[7]:
```
   SEQDOCUMENTO DATAPUBLICACAO TIPODOCUMENTO  NUMEROREGISTRO       PROCESSO  \
0     309631639     2025-05-05       DECISÃO    202000946481        TP 2695
1     310378377     2025-05-05       DECISÃO    202400362531  AREsp 2562174
2     309409286     2025-05-05       DECISÃO    202401080738   REsp 2132967

  DATARECEBIMENTO DATADISTRIBUICAO             NM_MINISTRO RECURSO  \
0      2020-04-25       2020-04-27   MARIA ISABEL GALLOTTI     NaN
1      2024-02-09       2024-02-28  JOÃO OTÁVIO DE NORONHA     NaN
2      2024-04-02       2024-04-04      MESSOD AZULAY NETO     NaN

          TEOR                                DESCRICAOMONOCRATICA  \
0     Negando  Prejudicada a ação de #{nome_da_parte} #{campo…
1     Negando  Conhecido o recurso de #{nome_da_parte} e não-…
2  Concedendo  Conhecido o recurso de #{nome_da_parte} e prov…

                  ASSUNTOS
0                     4993
1  11806, 7752, 7780, 7779
2                3416, 3435
```

[8]:
```
dfMetaDadosDecisao.
 ↪drop(columns=['DATAPUBLICACAO','TIPODOCUMENTO','DATARECEBIMENTO','DATADISTRIBUICAO','ASSUNT
 ↪inplace=True)
dfMetaDadosDecisao.head(3)
```

[8]:
```
   SEQDOCUMENTO  NUMEROREGISTRO       PROCESSO             NM_MINISTRO  \
0     309631639    202000946481        TP 2695   MARIA ISABEL GALLOTTI
1     310378377    202400362531  AREsp 2562174  JOÃO OTÁVIO DE NORONHA
2     309409286    202401080738   REsp 2132967      MESSOD AZULAY NETO

  RECURSO        TEOR                                DESCRICAOMONOCRATICA
0     NaN     Negando  Prejudicada a ação de #{nome_da_parte} #{campo…
1     NaN     Negando  Conhecido o recurso de #{nome_da_parte} e não-…
2     NaN  Concedendo  Conhecido o recurso de #{nome_da_parte} e prov…
```

[9]:
```
dfDetalheDecisao.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48027 entries, 0 to 48026
Data columns (total 6 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   SEQDOCUMENTO  48027 non-null  int64
 1   ACORDAO       0 non-null      float64
 2   EMENTA        13 non-null     object
 3   RELATORIO     0 non-null      float64
 4   VOTO          0 non-null      float64
 5   DECISAO       48011 non-null  object
```

```
dtypes: float64(3), int64(1), object(2)
memory usage: 2.2+ MB
```

[10]:
```
dfDetalheDecisao.head(3)
```

[10]:
```
   SEQDOCUMENTO  ACORDAO EMENTA  RELATORIO  VOTO  \
0     309631639      NaN    NaN        NaN   NaN
1     310378377      NaN    NaN        NaN   NaN
2     309409286      NaN    NaN        NaN   NaN

                                             DECISAO
0  O presente pedido de contracautela tinha por o…
1  Trata-se de agravo em recurso especial interpo…
2  Trata-se de recurso especial interposto por MA…
```

[11]:
```
dfDetalheDecisao.drop(columns=['ACORDAO', 'EMENTA', 'RELATORIO', 'VOTO'],␣
 ↪inplace=True)
dfDetalheDecisao.head(3)
```

[11]:
```
   SEQDOCUMENTO                                          DECISAO
0     309631639  O presente pedido de contracautela tinha por o…
1     310378377  Trata-se de agravo em recurso especial interpo…
2     309409286  Trata-se de recurso especial interposto por MA…
```

##Redução dos registros com o intuito de reduzir o tempo no treinamento e verificar a curva de convergência

###Separar o número de processo pelo tipo (Aresp, HC).

[12]:
```
#field = "AREsp 2562174".split(" ")
#print(f"Tipo: '{field[0]}' Processo: '{field[1]}'  ")

dfMetaDadosDecisao['TIPOPROCESSO'] = dfMetaDadosDecisao['PROCESSO'].
 ↪apply(lambda tipoprocesso: tipoprocesso.split(" ")[0])
dfMetaDadosDecisao['NUMEROPROCESSO'] = dfMetaDadosDecisao['PROCESSO'].
 ↪apply(lambda numeroprocesso: numeroprocesso.split(" ")[1])
dfMetaDadosDecisao.drop(columns=['PROCESSO'], inplace=True, axis=1)

dfMetaDadosDecisao.head(3)
```

[12]:
```
   SEQDOCUMENTO  NUMEROREGISTRO            NM_MINISTRO RECURSO        TEOR  \
0     309631639    202000946481   MARIA ISABEL GALLOTTI     NaN     Negando
1     310378377    202400362531  JOÃO OTÁVIO DE NORONHA     NaN     Negando
2     309409286    202401080738      MESSOD AZULAY NETO     NaN  Concedendo

                              DESCRICAOMONOCRATICA TIPOPROCESSO  \
0  Prejudicada a ação de #{nome_da_parte} #{campo…           TP
1  Conhecido o recurso de #{nome_da_parte} e não-…       AREsp
2  Conhecido o recurso de #{nome_da_parte} e prov…        REsp
```

```
  NUMEROPROCESSO
0           2695
1        2562174
2        2132967
```

[13]:
```
dfMetaDadosDecisao['TIPOPROCESSO'].value_counts()
```

[13]:
```
TIPOPROCESSO
AREsp          27718
HC              9123
REsp            6650
CC              1956
RHC             1534
RMS              349
EAREsp           309
Rcl              235
EREsp            122
CR               121
ExeMS             97
MS                89
PUIL              69
Pet               68
TutCautAnt        35
AR                29
TutAntAnt         23
RvCr              21
SLS                8
HD                 7
Ag                 6
TP                 5
HDE                5
EmbExeMS           4
Prc                2
SS                 2
ExeAR              1
Name: count, dtype: int64
```

[14]:
```
if ENVIRONMENT == 'COLAB':
  dfMetaDadosDecisao = dfMetaDadosDecisao[dfMetaDadosDecisao['TIPOPROCESSO'] ==␣
 ↪'REsp']
  dfMetaDadosDecisao.head(3)
```

##Preprocessing

###Noise removal

[15]:
```
import re

def remove_noise(text):
    if isinstance(text, str):
        clean = re.compile('<.*?>')
        return re.sub(clean, ' ', text)
    return text
```

[16]:
```
#Exemplo de uso da função
texto = """  <br><hr> Trata-se , de   habeas   corpus substitutivo   <b> de </b>
recurso          próprio , com pedido de liminar.Porém, tudo de bom.De bom.1.
 ↪1253,00</br> """
texto = remove_noise(texto)

print(texto)
```

```
     Trata-se , de   habeas   corpus substitutivo     de
recurso          próprio , com pedido de liminar.Porém, tudo de bom.De
bom.1.1253,00
```

[17]:
```
import re

def _consolidate_known_acronyms(text):
    # Consolidate 'art . ' into 'art.'
    text = re.sub(r'(?i)(\bart)\s+\.', r'\1.', text)

    # Consolidate 'n . ' into 'n.'
    text = re.sub(r'(?i)(\bn)\s+\.', r'\1.', text)

    # Consolidate 'c . c .' into 'c.c.'
    text = re.sub(r'(?i)(c)\s+\.\s*(c)\s+\.', r'\1.\2.', text)

    # Consolidate 'c . c' into 'c.c'
    text = re.sub(r'(?i)(c)\s+\.\s*(c)', r'\1.\2', text)

    # Consolidate 'c. c' into 'c.c'
    # This is for when the first dot is already correctly placed, but there's a␣
 ↪space before the second 'c'
    text = re.sub(r'(?i)(c\.)\s+(c)', r'\1\2', text)

    return text
```

[18]:
```
text = "583 dias-multa , incurso art. 33 , caput , c. c 40 , VI , Lei n . 11.
 ↪343/2006.Em sede recursal , Tribunal origem negou provimento apelo defensivo"
text = _consolidate_known_acronyms(text)
print(text)
```

```
583 dias-multa , incurso art. 33 , caput , c.c 40 , VI , Lei n. 11.343/2006.Em
```

sede recursal , Tribunal origem negou provimento apelo defensivo

###Stopword removal

[19]:
```
import re

def remove_whitespace(text):
    if not isinstance(text, str):
        return text

    # 1. Replace multiple whitespace characters with a single space and strip leading/trailing spaces
    text = re.sub(r'\s+', ' ', text).strip()

    # 2. Consolidate specific acronym patterns (e.g., 'art . ' -> 'art.')
    #text = _consolidate_known_acronyms(text)

    # 3. Remove space before commas (e.g., 'word , ' -> 'word,')
    text = re.sub(r'\s+,', ',', text)

    # 4. Handle dot spacing with a placeholder strategy for robustness:

    # Define unique placeholders for protected dots.
    PLACEHOLDER_ACRONYM_DOT_ART = "_#_ART_DOT_#_"
    PLACEHOLDER_ACRONYM_DOT_CC = "_#_CC_DOT_#_"
    PLACEHOLDER_ACRONYM_DOT_N = "_#_N_DOT_#_"
    PLACEHOLDER_DIGIT_DOT = "_#_DIGIT_DOT_#_"

    # --- Step 4.1: Protect dots that should NOT have a space after them ---
    # Order matters here: specific patterns first.

    # Protect dots preceded by digits AND followed by digits (e.g., 1.23)
    text = re.sub(r'(\d)\.(\d)', r'\1' + PLACEHOLDER_DIGIT_DOT + r'\2', text)

    # Protect 'art.' regardless of what follows (case-insensitive).
    text = re.sub(r'(?i)(\bart)\.', r'\1' + PLACEHOLDER_ACRONYM_DOT_ART, text)
    # Protect 'c.c.' regardless of what follows (case-insensitive).
    text = re.sub(r'(?i)(\bc\.c)\.', r'\1' + PLACEHOLDER_ACRONYM_DOT_CC, text)
    # Protect 'n.' regardless of what follows (case-insensitive).
    text = re.sub(r'(?i)(\bn)\.', r'\1' + PLACEHOLDER_ACRONYM_DOT_N, text)

    # --- Step 4.2: Apply general rule to add space after *remaining* dots ---
    # This will only affect dots that were not protected by the placeholders above.
    # It adds a space if a dot is followed by a non-space character.
    text = re.sub(r'\.(?=\S)', '. ', text)

    # --- Step 4.3: Restore protected dots ---
```

```
    # Replace placeholders back to original dots.
    text = text.replace(PLACEHOLDER_ACRONYM_DOT_ART, '.')
    text = text.replace(PLACEHOLDER_ACRONYM_DOT_CC, '.')
    text = text.replace(PLACEHOLDER_ACRONYM_DOT_N, '.')
    text = text.replace(PLACEHOLDER_DIGIT_DOT, '.')

    return text
```

[20]:
```
text = "<br>583 dias-multa , incurso art . 33 , <b>caput , </b>c. c 40 , VI ,␣
 ↪Lei n . 11.343/2006.Em sede recursal , Tribunal origem negou provimento␣
 ↪apelo defensivo."

text = remove_noise(text)
print(f"After remove_noise: {text}")

text = remove_whitespace(text)
print(f"After remove_whitespace: {text}")

text = _consolidate_known_acronyms(text)
print(f"After remove_whitespace: {text}")
```

```
After remove_noise:  583 dias-multa , incurso art . 33 ,  caput ,  c. c 40 , VI
, Lei n . 11.343/2006.Em sede recursal , Tribunal origem negou provimento apelo
defensivo.
After remove_whitespace: 583 dias-multa, incurso art . 33, caput, c. c 40, VI,
Lei n . 11.343/2006. Em sede recursal, Tribunal origem negou provimento apelo
defensivo.
After remove_whitespace: 583 dias-multa, incurso art. 33, caput, c.c 40, VI, Lei
n. 11.343/2006. Em sede recursal, Tribunal origem negou provimento apelo
defensivo.
```

[21]:
```
import nltk

nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')

stopwords = nltk.corpus.stopwords.words('portuguese')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     /home/alessandro.cporto/nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     /home/alessandro.cporto/nltk_data…
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     /home/alessandro.cporto/nltk_data…
[nltk_data]   Package punkt_tab is already up-to-date!
```

[22]:
```
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

def pre_processamento(texto):
    # Ensure texto is a string before processing
    if not isinstance(texto, str):
        return ""

    # Step 1: Remove HTML/XML tags (noise)
    texto = remove_noise(texto)

    # Step 2: Tokenize the text, keeping punctuation as separate tokens
    tokens = word_tokenize(texto, language='portuguese')

    # Step 3: Get Portuguese stopwords
    stop_words = set(stopwords.words('portuguese'))

    # Step 4: Filter out stopwords, keeping punctuation tokens and␣
 ↪non-alphabetic words
    # A token is kept if it's not a stopword (when lowercased) OR if it's not␣
 ↪purely alphabetic (e.g., punctuation, numbers)
    filtered_tokens = [w for w in tokens if w.lower() not in stop_words or not␣
 ↪w.isalpha()]

    # Step 5: Join the tokens back into a string
    texto = " ".join(filtered_tokens)

    # Step 6: Clean whitespace, consolidate acronyms, handle comma/dot spacing
    # The remove_whitespace function now internally calls the global␣
 ↪_consolidate_known_acronyms.
    texto = remove_whitespace(texto)

    texto = _consolidate_known_acronyms(texto)

    return texto
```

[23]:
```
#Exemplo de uso da função
texto = """<br>Trata-se de habeas corpus <b>substitutivo de recurso próprio</
 ↪b>, com pedido de liminar, impetrado em favor de LUIS FELIPE SILVA DE ASSIS,␣
 ↪em que se aponta com autoridade coatora o Tribunal de Justiça do Estado do␣
 ↪Rio de Janeiro.Consta dos autos que o paciente foi condenado à pena de 5␣
 ↪anos e 10 meses de reclusão, em regime semiaberto, mais pagamento de 583␣
 ↪dias-multa, como incurso no art. 33,caput, c.c o 40, VI, da Lei n. 11.343/
 ↪2006.Em sede recursal, o Tribunal de origem negou provimento ao apelo␣
 ↪defensivo."""
texto = pre_processamento(texto)
```

```
print(texto)
```

```
Trata-se habeas corpus substitutivo recurso próprio, pedido liminar, impetrado
favor LUIS FELIPE SILVA ASSIS, aponta autoridade coatora Tribunal Justiça Estado
Rio Janeiro. Consta autos paciente condenado pena 5 anos 10 meses reclusão,
regime semiaberto, pagamento 583 dias-multa, incurso art. 33, caput, c.c 40, VI,
Lei n. 11.343/2006. Em sede recursal, Tribunal origem negou provimento apelo
defensivo .
```

[24]:
```
dfDetalheDecisao['DECISAO'] = dfDetalheDecisao['DECISAO'].
 ↪apply(pre_processamento)
```

###Junção dos Datasets em um único dataset

[25]:
```
def get_decisao(seq_documento):
    # Filter dfDetalheAcordaos for the given SEQDOCUMENTO
    decisao_row = dfDetalheDecisao[dfDetalheDecisao['SEQDOCUMENTO'] ==␣
 ↪seq_documento]

    # Check if the filtered DataFrame is empty
    if not decisao_row.empty:
        # If not empty, return the 'ACORDAO' value
        return decisao_row.head(1)['DECISAO'].iloc[0]
    else:
        # If empty, return None
        return None

def get_tipo_decisao(teor_decisao):
    # Filter dfDetalheAcordaos for the given SEQDOCUMENTO
    tipo_decisao_row = dfTipoDecisao[dfTipoDecisao['DESC_TEOR_DECISAO'] ==␣
 ↪teor_decisao].head(1)

    # Check if the filtered DataFrame is empty
    if not tipo_decisao_row.empty:
        # If not empty, return the 'ACORDAO' value
        return tipo_decisao_row.head(1)['DECISAO_FAVORAVEL'].iloc[0]
    else:
        # If empty, return None
        return None
```

[26]:
```
dfMetaDadosDecisao['DECISAO']  = dfMetaDadosDecisao['SEQDOCUMENTO'].
 ↪apply(get_decisao)
dfMetaDadosDecisao['DECISAO_FAVORAVEL']  = dfMetaDadosDecisao['TEOR'].
 ↪apply(get_tipo_decisao)
```

[27]:
```
dfMetaDadosDecisao.head(3)
```

[27]:
```
   SEQDOCUMENTO  NUMEROREGISTRO             NM_MINISTRO RECURSO        TEOR  \
0     309631639    202000946481   MARIA ISABEL GALLOTTI     NaN     Negando
1     310378377    202400362531  JOÃO OTÁVIO DE NORONHA     NaN     Negando
2     309409286    202401080738      MESSOD AZULAY NETO     NaN  Concedendo

                                DESCRICAOMONOCRATICA TIPOPROCESSO  \
0  Prejudicada a ação de #{nome_da_parte} #{campo…           TP
1  Conhecido o recurso de #{nome_da_parte} e não-…        AREsp
2  Conhecido o recurso de #{nome_da_parte} e prov…         REsp

  NUMEROPROCESSO                                            DECISAO  \
0           2695  presente pedido contracautela objeto suspensão…
1        2562174  Trata-se agravo recurso especial interposto MI…
2        2132967  Trata-se recurso especial interposto MAYCKON S…

  DECISAO_FAVORAVEL
0                 N
1                 N
2                 S
```

[28]:
```
# Filter rows where 'TEOR' is not empty (NaN) and assign to a new dataframe
dfMetaDadosDecisao_filtered = dfMetaDadosDecisao.dropna(subset=['TEOR']).copy()

# Filter rows where 'DECISAO' is not empty (NaN) and assign to a new dataframe
dfMetaDadosDecisao_filtered = dfMetaDadosDecisao_filtered.
 ↪dropna(subset=['DECISAO']).copy()

# Filter rows where 'DECISAO_FAVORAVEL is not empty (NaN) and assign to a new␣
 ↪dataframe -- decisões neutras não possuem efeito jurídico considerável
dfMetaDadosDecisao_filtered = dfMetaDadosDecisao_filtered.
 ↪dropna(subset=['DECISAO_FAVORAVEL']).copy()

dfMetaDadosDecisao = dfMetaDadosDecisao_filtered
dfMetaDadosDecisao.head(5)
```

[28]:
```
   SEQDOCUMENTO  NUMEROREGISTRO  \
0     309631639    202000946481
1     310378377    202400362531
2     309409286    202401080738
3     310313783    202403756009
4     308791735    202500051465

                                        NM_MINISTRO RECURSO            TEOR  \
0                            MARIA ISABEL GALLOTTI     NaN         Negando
1                           JOÃO OTÁVIO DE NORONHA     NaN         Negando
2                               MESSOD AZULAY NETO     NaN      Concedendo
3  CARLOS CINI MARCHIONATTI (DESEMBARGADOR CONVOC…     NaN  Não Conhecendo
```

```
4                              ROGERIO SCHIETTI CRUZ     NaN          Negando

                                DESCRICAOMONOCRATICA TIPOPROCESSO  \
0  Prejudicada a ação de #{nome_da_parte} #{campo…           TP
1  Conhecido o recurso de #{nome_da_parte} e não-…        AREsp
2  Conhecido o recurso de #{nome_da_parte} e prov…         REsp
3  Não conhecido o recurso de #{nome_da_parte} #{…        AREsp
4  Denegado o Habeas Corpus de #{nome_da_parte} #…           HC

  NUMEROPROCESSO                                          DECISAO  \
0           2695  presente pedido contracautela objeto suspensão…
1        2562174  Trata-se agravo recurso especial interposto MI…
2        2132967  Trata-se recurso especial interposto MAYCKON S…
3        2761478  agravo interposto contra decisão Tribunal Just…
4         973936  JOSIMAR TEOTONIO PONTES alega vítima coação il…

  DECISAO_FAVORAVEL
0                 N
1                 N
2                 S
3                 N
4                 N
```

[29]:
```
dfMetaDadosDecisao.head()
```

[29]:
```
   SEQDOCUMENTO  NUMEROREGISTRO  \
0     309631639    202000946481
1     310378377    202400362531
2     309409286    202401080738
3     310313783    202403756009
4     308791735    202500051465

                                        NM_MINISTRO RECURSO            TEOR  \
0                             MARIA ISABEL GALLOTTI     NaN         Negando
1                            JOÃO OTÁVIO DE NORONHA     NaN         Negando
2                               MESSOD AZULAY NETO     NaN      Concedendo
3  CARLOS CINI MARCHIONATTI (DESEMBARGADOR CONVOC…     NaN  Não Conhecendo
4                              ROGERIO SCHIETTI CRUZ     NaN         Negando

                                DESCRICAOMONOCRATICA TIPOPROCESSO  \
0  Prejudicada a ação de #{nome_da_parte} #{campo…           TP
1  Conhecido o recurso de #{nome_da_parte} e não-…        AREsp
2  Conhecido o recurso de #{nome_da_parte} e prov…         REsp
3  Não conhecido o recurso de #{nome_da_parte} #{…        AREsp
4  Denegado o Habeas Corpus de #{nome_da_parte} #…           HC

  NUMEROPROCESSO                                          DECISAO  \
```

```
0          2695  presente pedido contracautela objeto suspensão…
1       2562174  Trata-se agravo recurso especial interposto MI…
2       2132967  Trata-se recurso especial interposto MAYCKON S…
3       2761478  agravo interposto contra decisão Tribunal Just…
4        973936  JOSIMAR TEOTONIO PONTES alega vítima coação il…

  DECISAO_FAVORAVEL
0                 N
1                 N
2                 S
3                 N
4                 N
```

0.0.1 Junção dos campos DESCRICAOMONOCRATICA com DECISAO

Estudar a possibilidade de dar maior atenção à decisão monocrática.

[30]:
```
dfMetaDadosDecisao.head(3)
```

[30]:
```
   SEQDOCUMENTO  NUMEROREGISTRO             NM_MINISTRO RECURSO        TEOR  \
0     309631639    202000946481   MARIA ISABEL GALLOTTI     NaN     Negando
1     310378377    202400362531  JOÃO OTÁVIO DE NORONHA     NaN     Negando
2     309409286    202401080738      MESSOD AZULAY NETO     NaN  Concedendo

                              DESCRICAOMONOCRATICA TIPOPROCESSO  \
0  Prejudicada a ação de #{nome_da_parte} #{campo…           TP
1  Conhecido o recurso de #{nome_da_parte} e não-…       AREsp
2  Conhecido o recurso de #{nome_da_parte} e prov…        REsp

  NUMEROPROCESSO                                           DECISAO  \
0          2695  presente pedido contracautela objeto suspensão…
1       2562174  Trata-se agravo recurso especial interposto MI…
2       2132967  Trata-se recurso especial interposto MAYCKON S…

  DECISAO_FAVORAVEL
0                 N
1                 N
2                 S
```

##Análise de dados após tratamento

[31]:
```
dfMetaDadosDecisao['DECISAOCOMPLETA'] =␣
↪dfMetaDadosDecisao['DESCRICAOMONOCRATICA'] + ' -- ' +␣
↪dfMetaDadosDecisao['DECISAO']
```

###Verificando distribuição de dados entre as decisões favoráveis ou não ao réu

####Consulta direta no tipo de decisão (Favorável ou não)

[32]:
```
#dfMetaDadosDecisao['DECISAOCOMPLETA'][283]
```

[33]:
```
dfMetaDadosDecisao_filtered[dfMetaDadosDecisao_filtered['DECISAO_FAVORAVEL'] ␣
↪== 'N'].head(5)
```

[33]:

	SEQDOCUMENTO	NUMEROREGISTRO	NM_MINISTRO	RECURSO	TEOR	DESCRICAOMONOCRATICA	TIPOPROCESSO	NUMEROPROCESSO	DECISAO	DECISAO_FAVORAVEL	DECISAOCOMPLETA
0	309631639	202000946481	MARIA ISABEL GALLOTTI	NaN	Negando	Prejudicada a ação de #{nome_da_parte} #{campo…	TP	2695	presente pedido contracautela objeto suspensão…	N	Prejudicada a ação de #{nome_da_parte} #{campo…
1	310378377	202400362531	JOÃO OTÁVIO DE NORONHA	NaN	Negando	Conhecido o recurso de #{nome_da_parte} e não-…	AREsp	2562174	Trata-se agravo recurso especial interposto MI…	N	Conhecido o recurso de #{nome_da_parte} e não-…
3	310313783	202403756009	CARLOS CINI MARCHIONATTI (DESEMBARGADOR CONVOC…	NaN	Não Conhecendo	Não conhecido o recurso de #{nome_da_parte} #{…	AREsp	2761478	agravo interposto contra decisão Tribunal Just…	N	Não conhecido o recurso de #{nome_da_parte} #{…
4	308791735	202500051465	ROGERIO SCHIETTI CRUZ	NaN	Negando	Denegado o Habeas Corpus de #{nome_da_parte} #…	HC	973936	JOSIMAR TEOTONIO PONTES alega vítima coação il…	N	Denegado o Habeas Corpus de #{nome_da_parte} #…
5	310057660	202501165811	HERMAN BENJAMIN	NaN	Não Conhecendo	Não conhecido o recurso de #{nome_da_parte} #{…	AREsp	2899900	Cuida-se Agravo Recurso Especial apresentado L…	N	Não conhecido o recurso de #{nome_da_parte} #{…

[34]:
```
dfMetaDadosDecisao_filtered.shape
```

[34]:
```
(48027, 11)
```

####Verificando distribuição dos dados propriamente dita

[35]:
```
dataset = dfMetaDadosDecisao
```

```
total   = dataset.shape[0]
label_s = dfMetaDadosDecisao_filtered[dataset['DECISAO_FAVORAVEL']  == 'S'].
 ↪shape[0]
label_n = dfMetaDadosDecisao_filtered[dataset['DECISAO_FAVORAVEL']  == 'N'].
 ↪shape[0]

print(f'TOTAL  : {total}')
print(f'LABEL S: {label_s} - % {round(label_s/total*100, 2)}')
print(f'LABEL N: {label_n} - % {round(label_n/total*100, 2)}')
```

```
TOTAL  : 48027
LABEL S: 7402 - % 15.41
LABEL N: 40625 - % 84.59
```

##Preparação dos dados para Treinamento

###Separando entre dados de treinamento e teste

[36]:
```
from sklearn.model_selection import train_test_split

# Separate features (X) and target (y)
X = dataset.drop('DECISAO_FAVORAVEL', axis=1)
y = dataset['DECISAO_FAVORAVEL']

# Split the data into training and testing sets, stratifying by␣
 ↪'DECISAO_FAVORAVEL'
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.35,␣
 ↪random_state=42, stratify=y)

print("Training set shape (X_train, y_train):", X_train.shape, y_train.shape)
print("Testing set shape (X_test, y_test):", X_test.shape, y_test.shape)

# Display the distribution of 'DECISAO_FAVORAVEL' in the original dataset,␣
 ↪training set, and testing set
print("\nOriginal dataset 'DECISAO_FAVORAVEL' distribution:")
display(y.value_counts(normalize=True))

print("\nTraining set 'DECISAO_FAVORAVEL' distribution:")
display(y_train.value_counts(normalize=True))

print("\nTesting set 'DECISAO_FAVORAVEL' distribution:")
display(y_test.value_counts(normalize=True))
```

```
Training set shape (X_train, y_train): (31217, 10) (31217,)
Testing set shape (X_test, y_test): (16810, 10) (16810,)

Original dataset 'DECISAO_FAVORAVEL' distribution:

DECISAO_FAVORAVEL
```

```
N    0.845878
S    0.154122
Name: proportion, dtype: float64


Training set 'DECISAO_FAVORAVEL' distribution:

DECISAO_FAVORAVEL
N    0.845885
S    0.154115
Name: proportion, dtype: float64


Testing set 'DECISAO_FAVORAVEL' distribution:

DECISAO_FAVORAVEL
N    0.845866
S    0.154134
Name: proportion, dtype: float64
```

[37]:
```
len(X['DECISAO'][2])
```

[37]:
```
5533
```

1 Task

Implement and evaluate a BERT-based sequence classification model (`CustomBertForSequenceClassificationWithMeanPooling`) that incorporates a linear projection layer for dimensionality reduction. The model's configuration will be updated to include a `reduced_embedding_dim` parameter, and the custom model will be modified to apply this linear projection after mean pooling, before the final classification head. The task includes instantiating the model with the new configuration, training it using the `Trainer`, evaluating its performance, and generating a confusion matrix, ROC curve, and training history plots. Finally, summarize the implementation details of the linear projection layer and present the evaluation results.

[38]:
```
#2
import torch.nn as nn
from transformers import BertPreTrainedModel, BertModel
from transformers.modeling_outputs import SequenceClassifierOutput

class CustomBertForSequenceClassificationWithMeanPooling(BertPreTrainedModel):
    def __init__(self, config):
        super().__init__(config)
        self.num_labels = config.num_labels

        self.bert = BertModel(config)

        # Linear projection layer for dimensionality reduction
```

```
        self.linear_projection = nn.Linear(config.hidden_size, config.
↪reduced_embedding_dim)
        self.activation = nn.GELU() # Using GELU as a common activation function

        # Classification head, taking input from the reduced embedding dimension
        self.classifier = nn.Linear(config.reduced_embedding_dim, config.
↪num_labels)

        # Initialize weights and apply final processing
        self.post_init()

    def _freeze_layers(self, num_layers_to_freeze):
        # Freeze embeddings
        for param in self.bert.embeddings.parameters():
            param.requires_grad = False

        # Freeze the first `num_layers_to_freeze` of the encoder
        for i, layer in enumerate(self.bert.encoder.layer):
            if i < num_layers_to_freeze:
                for param in layer.parameters():
                    param.requires_grad = False
            else:
                break
        print(f"Froze {num_layers_to_freeze} BERT encoder layers and embeddings.
↪")

    def forward(
        self,
        input_ids=None,
        attention_mask=None,
        token_type_ids=None,
        labels=None,
        output_hidden_states=None,
        return_dict=None
    ):
        return_dict = return_dict if return_dict is not None else self.config.
↪use_return_dict

        outputs = self.bert(
            input_ids,
            attention_mask=attention_mask,
            token_type_ids=token_type_ids,
            output_hidden_states=False, # Changed to False to save memory␣
↪during evaluation
            output_attentions=False # Changed to False to save memory during␣
↪evaluation
```

```
        )

        # Mean pooling of the last hidden state (all tokens)
        last_hidden_state = outputs.last_hidden_state
        # Apply attention mask to consider only actual tokens
        input_mask_expanded = attention_mask.unsqueeze(-1).
↪expand(last_hidden_state.size()).float()
        sum_embeddings = torch.sum(last_hidden_state * input_mask_expanded, 1)
        sum_mask = torch.clamp(input_mask_expanded.sum(1), min=1e-9)
        pooled_output = sum_embeddings / sum_mask

        # Apply linear projection and activation
        projected_output = self.linear_projection(pooled_output)
        activated_output = self.activation(projected_output)

        # Pass through the classifier head
        logits = self.classifier(activated_output)

        loss = None
        if labels is not None:
            loss_fct = nn.CrossEntropyLoss()
            loss = loss_fct(logits.view(-1, self.num_labels), labels.view(-1))

        if not return_dict:
            # Return only logits if not using return_dict and other outputs are␣
↪not needed
            output = (logits,)
            return ((loss,) + output) if loss is not None else output

        return SequenceClassifierOutput(
            loss=loss,
            logits=logits,
            # Do not pass hidden_states and attentions if they are None to save␣
↪memory
            # hidden_states=outputs.hidden_states,
            # attentions=outputs.attentions,
        )

print("CustomBertForSequenceClassificationWithMeanPooling class defined.")
```

```
2025-11-23 19:54:13.612161: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-11-23 19:54:13.688457: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
```

```
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other
operations, rebuild TensorFlow with the appropriate compiler flags.
2025-11-23 19:54:16.625351: I tensorflow/core/util/port.cc:153] oneDNN custom
operations are on. You may see slightly different numerical results due to
floating-point round-off errors from different computation orders. To turn them
off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.

CustomBertForSequenceClassificationWithMeanPooling class defined.
```

1.1 Update Model Configuration

1.1.1 Subtask:

Add a new parameter, `reduced_embedding_dim`, to the model's configuration. This will define the target dimensionality after the linear projection layer. This step ensures that the configuration object passed to the custom model contains the necessary information for the new layer.

[39]:
```python
from transformers import AutoConfig

# Load the pre-trained configuration
config = AutoConfig.from_pretrained(model_checkpoint)

# Add the new parameter to the configuration
config.reduced_embedding_dim = 512

print(f"Model configuration loaded and updated. Reduced embedding dimension set␣
↪to: {config.reduced_embedding_dim}")
```

```
Model configuration loaded and updated. Reduced embedding dimension set to: 512
```

1.2 Instantiate Custom Model with New Configuration

1.2.1 Subtask:

Re-instantiate the CustomBertForSequenceClassificationWithMeanPooling model, passing the updated configuration (including reduced_embedding_dim) and ensuring the model is moved to the appropriate device (CUDA if available). The new linear projection layer will now be part of the model.

[40]:
```python
model_labels = y.nunique()
config.num_labels = model_labels

# Re-instantiate the model with the updated config
model = CustomBertForSequenceClassificationWithMeanPooling.
↪from_pretrained(model_checkpoint, config=config)
model.to(device)
model._freeze_layers(num_layers_to_freeze)

print("Model re-instantiated with updated configuration and layers frozen.")
```

```
Some weights of CustomBertForSequenceClassificationWithMeanPooling were not
initialized from the model checkpoint at neuralmind/bert-large-portuguese-cased
and are newly initialized: ['classifier.bias', 'classifier.weight',
'linear_projection.bias', 'linear_projection.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

Froze 165 BERT encoder layers and embeddings.
Model re-instantiated with updated configuration and layers frozen.
```

[41]:
```python
from transformers import AutoTokenizer

# Initialize the tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)

# Map 'S' to 1 and 'N' to 0 for the labels
label_map = {'S': 1, 'N': 0}
y_train_encoded = y_train.map(label_map).tolist()
y_test_encoded = y_test.map(label_map).tolist()

# Tokenize the training and test datasets, explicitly setting max_length
# A common max_length for BERT models is 512
train_encodings = tokenizer(X_train['DECISAOCOMPLETA'].tolist(),␣
 ↪truncation=True, padding=True, max_length=512)
test_encodings = tokenizer(X_test['DECISAOCOMPLETA'].tolist(), truncation=True,␣
 ↪padding=True, max_length=512)

print("Tokenizer initialized and datasets tokenized.")
print(f"Number of training samples: {len(train_encodings['input_ids'])}")
print(f"Number of testing samples: {len(test_encodings['input_ids'])}")
```

```
Tokenizer initialized and datasets tokenized.
Number of training samples: 31217
Number of testing samples: 16810
```

[42]:
```python
import torch

class CustomDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.
 ↪items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item
```

```
    def __len__(self):
        return len(self.labels)

# Instantiate the custom Dataset for training and testing
train_dataset = CustomDataset(train_encodings, y_train_encoded)
test_dataset = CustomDataset(test_encodings, y_test_encoded)

print("CustomDataset class defined and datasets instantiated.")
print(f"Train dataset size: {len(train_dataset)}")
print(f"Test dataset size: {len(test_dataset)}")
```

```
CustomDataset class defined and datasets instantiated.
Train dataset size: 31217
Test dataset size: 16810
```

[43]:
```
from transformers import TrainingArguments

# Define output directory
output_dir = './results'

# Create TrainingArguments object
training_args = TrainingArguments(
    output_dir=output_dir,
    num_train_epochs=EPOCHS,
    per_device_train_batch_size=4, # Further reduced from 8 to 4
    per_device_eval_batch_size=4,  # Further reduced from 8 to 4
    warmup_steps=500,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=10,
    eval_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
    report_to=[], # Changed from 'none' to [] to correctly disable reporting
    fp16=True, # Enable mixed-precision training
    gradient_checkpointing=True # Enable gradient checkpointing for memory␣
 ↪optimization
)

print("TrainingArguments configured with further reduced batch sizes, fp16,␣
 ↪gradient checkpointing enabled, and reporting disabled.")
```

```
TrainingArguments configured with further reduced batch sizes, fp16, gradient
checkpointing enabled, and reporting disabled.
```

1.3 Initialize and Train Trainer

1.3.1 Subtask:

Initialize the Trainer with the modified custom model, the tokenized datasets, and the configured training arguments. Then, execute the trainer.train() method to start the training process.

[44]:
```
if not install_evaluate:
  !pip install evaluate
  install_evaluate = True
```

```
huggingface/tokenizers: The current process just got forked, after parallelism
has already been used. Disabling parallelism to avoid deadlocks…
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true |
false)
```

```
Requirement already satisfied: evaluate in ./venv/lib/python3.12/site-packages
(0.4.6)
Requirement already satisfied: datasets>=2.0.0 in ./venv/lib/python3.12/site-
packages (from evaluate) (4.3.0)
Requirement already satisfied: numpy>=1.17 in ./venv/lib/python3.12/site-
packages (from evaluate) (2.3.4)
Requirement already satisfied: dill in ./venv/lib/python3.12/site-packages (from
evaluate) (0.4.0)
Requirement already satisfied: pandas in ./venv/lib/python3.12/site-packages
(from evaluate) (2.3.3)
Requirement already satisfied: requests>=2.19.0 in ./venv/lib/python3.12/site-
packages (from evaluate) (2.32.5)
Requirement already satisfied: tqdm>=4.62.1 in ./venv/lib/python3.12/site-
packages (from evaluate) (4.67.1)
Requirement already satisfied: xxhash in ./venv/lib/python3.12/site-packages
(from evaluate) (3.6.0)
Requirement already satisfied: multiprocess in ./venv/lib/python3.12/site-
packages (from evaluate) (0.70.16)
Requirement already satisfied: fsspec>=2021.05.0 in ./venv/lib/python3.12/site-
packages (from fsspec[http]>=2021.05.0->evaluate) (2025.9.0)
Requirement already satisfied: huggingface-hub>=0.7.0 in
./venv/lib/python3.12/site-packages (from evaluate) (0.36.0)
Requirement already satisfied: packaging in ./venv/lib/python3.12/site-packages
(from evaluate) (25.0)
Requirement already satisfied: filelock in ./venv/lib/python3.12/site-packages
(from datasets>=2.0.0->evaluate) (3.19.1)
Requirement already satisfied: pyarrow>=21.0.0 in ./venv/lib/python3.12/site-
packages (from datasets>=2.0.0->evaluate) (22.0.0)
Requirement already satisfied: httpx<1.0.0 in ./venv/lib/python3.12/site-
packages (from datasets>=2.0.0->evaluate) (0.28.1)
Requirement already satisfied: pyyaml>=5.1 in ./venv/lib/python3.12/site-
packages (from datasets>=2.0.0->evaluate) (6.0.3)
```

```
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in
./venv/lib/python3.12/site-packages (from fsspec[http]>=2021.05.0->evaluate)
(3.13.2)
Requirement already satisfied: typing-extensions>=3.7.4.3 in
./venv/lib/python3.12/site-packages (from huggingface-hub>=0.7.0->evaluate)
(4.15.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in
./venv/lib/python3.12/site-packages (from huggingface-hub>=0.7.0->evaluate)
(1.2.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
./venv/lib/python3.12/site-packages (from requests>=2.19.0->evaluate) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in ./venv/lib/python3.12/site-
packages (from requests>=2.19.0->evaluate) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in ./venv/lib/python3.12/site-
packages (from requests>=2.19.0->evaluate) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in ./venv/lib/python3.12/site-
packages (from requests>=2.19.0->evaluate) (2025.10.5)
Requirement already satisfied: python-dateutil>=2.8.2 in
./venv/lib/python3.12/site-packages (from pandas->evaluate) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in ./venv/lib/python3.12/site-
packages (from pandas->evaluate) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in ./venv/lib/python3.12/site-
packages (from pandas->evaluate) (2025.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in
./venv/lib/python3.12/site-packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in ./venv/lib/python3.12/site-
packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate)
(1.4.0)
Requirement already satisfied: attrs>=17.3.0 in ./venv/lib/python3.12/site-
packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate)
(25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in ./venv/lib/python3.12/site-
packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate)
(1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in
./venv/lib/python3.12/site-packages (from
aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate) (6.7.0)
Requirement already satisfied: propcache>=0.2.0 in ./venv/lib/python3.12/site-
packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate)
(0.4.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in ./venv/lib/python3.12/site-
packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]>=2021.05.0->evaluate)
(1.22.0)
Requirement already satisfied: anyio in ./venv/lib/python3.12/site-packages
(from httpx<1.0.0->datasets>=2.0.0->evaluate) (4.11.0)
Requirement already satisfied: httpcore==1.* in ./venv/lib/python3.12/site-
packages (from httpx<1.0.0->datasets>=2.0.0->evaluate) (1.0.9)
```

```
Requirement already satisfied: h11>=0.16 in ./venv/lib/python3.12/site-packages
(from httpcore==1.*->httpx<1.0.0->datasets>=2.0.0->evaluate) (0.16.0)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages
(from python-dateutil>=2.8.2->pandas->evaluate) (1.17.0)
Requirement already satisfied: sniffio>=1.1 in ./venv/lib/python3.12/site-
packages (from anyio->httpx<1.0.0->datasets>=2.0.0->evaluate) (1.3.1)
```

[45]:
```python
from transformers import Trainer
import evaluate
import numpy as np

# Load evaluate metrics
accuracy_metric = evaluate.load("accuracy")
precision_metric = evaluate.load("precision")
recall_metric = evaluate.load("recall")
f1_metric = evaluate.load("f1")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    accuracy = accuracy_metric.compute(predictions=predictions,␣
 ↪references=labels)
    precision = precision_metric.compute(predictions=predictions,␣
 ↪references=labels, average='weighted')
    recall = recall_metric.compute(predictions=predictions, references=labels,␣
 ↪average='weighted')
    f1 = f1_metric.compute(predictions=predictions, references=labels,␣
 ↪average='weighted')

    return {
        "accuracy": accuracy["accuracy"],
        "precision": precision["precision"],
        "recall": recall["recall"],
        "f1": f1["f1"],
    }

# Instantiate the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)

# Start training
```

```
print("Starting model training...")
trainer.train()
print("Model training completed.")
```

```
Downloading builder script: 0.00B [00:00, ?B/s]

Downloading builder script: 0.00B [00:00, ?B/s]

Downloading builder script: 0.00B [00:00, ?B/s]

Starting model training…

/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(

<IPython.core.display.HTML object>

/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
```

```
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
```

```
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
```

```
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
```

```
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
```

```
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
```

```
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
```

```
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
```

```
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(
/home/alessandro.cporto/projeto_llm_pucpr/venv/lib/python3.12/site-
packages/torch/utils/checkpoint.py:85: UserWarning: None of the inputs have
requires_grad=True. Gradients will be None
  warnings.warn(

Model training completed.
```

[47]:
```
!pip install seaborn
```

```
huggingface/tokenizers: The current process just got forked, after parallelism
has already been used. Disabling parallelism to avoid deadlocks…
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true |
false)

Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in
./venv/lib/python3.12/site-packages (from seaborn) (2.3.4)
Requirement already satisfied: pandas>=1.2 in ./venv/lib/python3.12/site-
packages (from seaborn) (2.3.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in
./venv/lib/python3.12/site-packages (from seaborn) (3.10.7)
Requirement already satisfied: contourpy>=1.0.1 in ./venv/lib/python3.12/site-
```

```
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./venv/lib/python3.12/site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./venv/lib/python3.12/site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in ./venv/lib/python3.12/site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.9)
Requirement already satisfied: packaging>=20.0 in ./venv/lib/python3.12/site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (25.0)
Requirement already satisfied: pillow>=8 in ./venv/lib/python3.12/site-packages
(from matplotlib!=3.6.1,>=3.4->seaborn) (12.0.0)
Requirement already satisfied: pyparsing>=3 in ./venv/lib/python3.12/site-
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in
./venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in ./venv/lib/python3.12/site-
packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in ./venv/lib/python3.12/site-
packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages
(from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)

294.9/294.9 kB 4.5 MB/s eta 0:00:00[31m5.2 MB/s
eta 0:00:01
Installing collected packages: seaborn
Successfully installed seaborn-0.13.2
```

[48]:
```python
#2
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, roc_curve, auc

# Get predictions on the test set
print("Getting predictions on the test set...")
predictions = trainer.predict(test_dataset)

# Extract logits and true labels
logits = predictions.predictions
y_pred_raw = np.argmax(logits, axis=-1)
y_true = predictions.label_ids

# Compute metrics using the compute_metrics function with correct arguments
metrics = compute_metrics((predictions.predictions, predictions.label_ids))
print("Evaluation Metrics:", metrics)
```

```
# Confusion Matrix
cm = confusion_matrix(y_true, y_pred_raw)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['N', 'S'],␣
 ↪yticklabels=['N', 'S'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# ROC Curve
# Ensure y_scores are probabilities for the positive class
y_scores = logits[:, 1] # Probability of class 'S' (positive class)
fpr, tpr, thresholds = roc_curve(y_true, y_scores)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:
 ↪.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()

# Training History Plots
# trainer.state.log_history contains training and evaluation logs
history = trainer.state.log_history

# Filter for relevant entries for training loss (should only be steps where␣
 ↪'loss' is present but not 'eval_loss')
train_loss_entries = [entry for entry in history if 'loss' in entry and␣
 ↪'eval_loss' not in entry]
train_loss = [entry['loss'] for entry in train_loss_entries]
steps = [entry['step'] for entry in train_loss_entries]

# Filter for relevant entries for evaluation metrics (should contain␣
 ↪'eval_loss' or other eval metrics)
eval_entries = [entry for entry in history if 'eval_loss' in entry]
eval_loss = [entry['eval_loss'] for entry in eval_entries]
eval_steps = [entry['step'] for entry in eval_entries]

# Ensure these lists are populated correctly for plotting
if eval_entries:
```

```
    eval_accuracy = [entry['eval_accuracy'] for entry in eval_entries]
    eval_precision = [entry['eval_precision'] for entry in eval_entries]
    eval_recall = [entry['eval_recall'] for entry in eval_entries]
    eval_f1 = [entry['eval_f1'] for entry in eval_entries]
    epochs = range(1, len(eval_loss) + 1) # Assuming eval_loss is recorded per epoch
else:
    print("No evaluation history found for plotting metrics.")
    eval_accuracy, eval_precision, eval_recall, eval_f1, epochs = [], [], [], [], []

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(steps, train_loss, label='Training Loss')
# Only plot validation loss if there is evaluation history
if eval_loss:
    plt.plot(eval_steps, eval_loss, label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Steps')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2)
# Only plot validation metrics if there is evaluation history
if eval_accuracy:
    plt.plot(epochs, eval_accuracy, label='Validation Accuracy')
    plt.plot(epochs, eval_precision, label='Validation Precision')
    plt.plot(epochs, eval_recall, label='Validation Recall')
    plt.plot(epochs, eval_f1, label='Validation F1-Score')
    plt.title('Validation Metrics over Epochs')
    plt.xlabel('Epochs')
    plt.ylabel('Score')
    plt.legend()
else:
    plt.text(0.5, 0.5, 'No validation metrics to display', horizontalalignment='center', verticalalignment='center', transform=plt.gca().transAxes)
    plt.title('Validation Metrics over Epochs')

plt.tight_layout()
plt.show()

# Summarize implementation of linear projection layer and evaluation results
summary = f"""
Implementation Details of Linear Projection Layer:
```

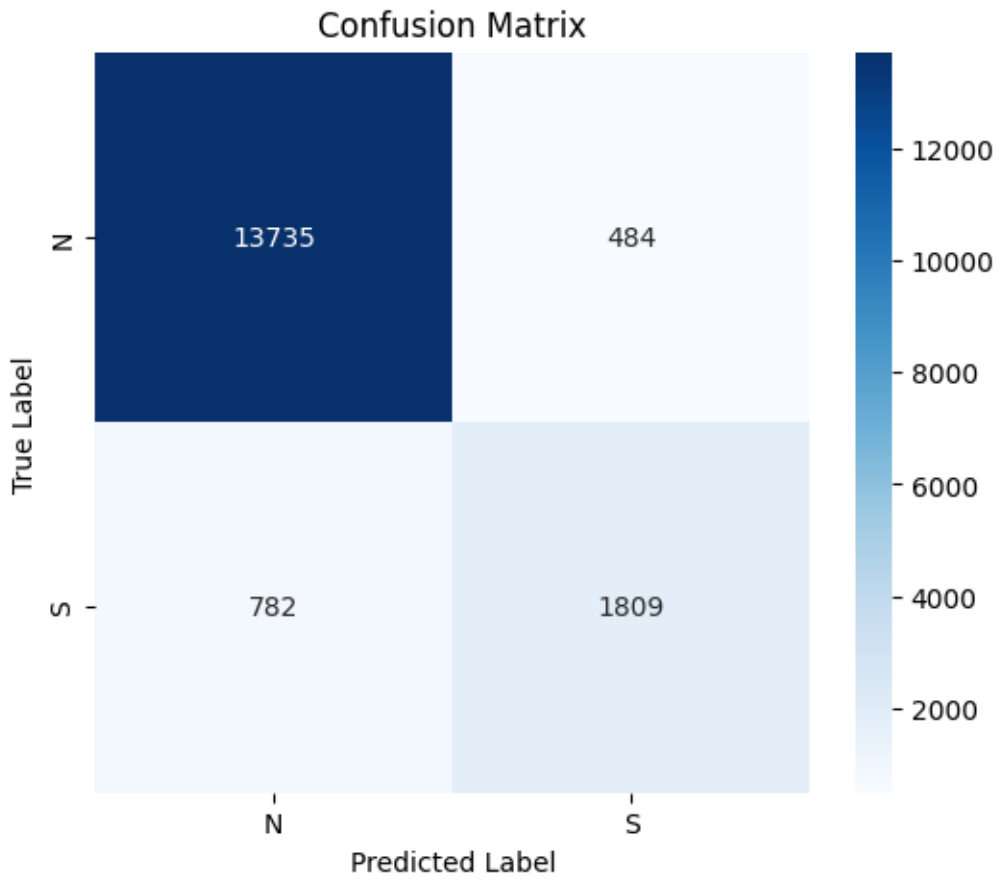
```
- A `linear_projection` layer (nn.Linear) was added after the mean pooling step␣
 ↪in `CustomBertForSequenceClassificationWithMeanPooling`.
- It reduces the dimensionality of the pooled embeddings from `config.
 ↪hidden_size` (1024 for bert-large-portuguese-cased) to `config.
 ↪reduced_embedding_dim` (256).
- A GELU activation function is applied after the linear projection.
- The final classification head (`self.classifier`) then takes this␣
 ↪256-dimensional vector as input.
- This approach aims to reduce the number of parameters in the classification␣
 ↪head, potentially speeding up training and reducing the risk of overfitting,␣
 ↪especially when freezing most of the base BERT model.

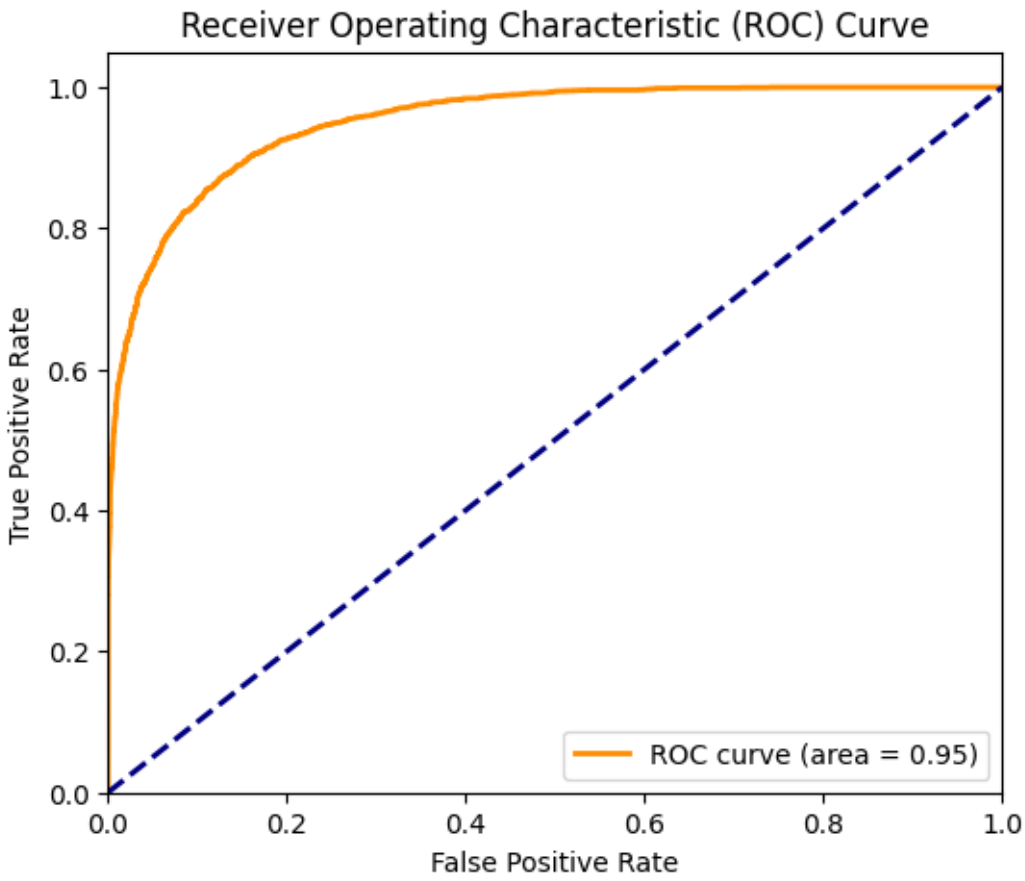
Evaluation Results:
{metrics}
"""
print(summary)
```

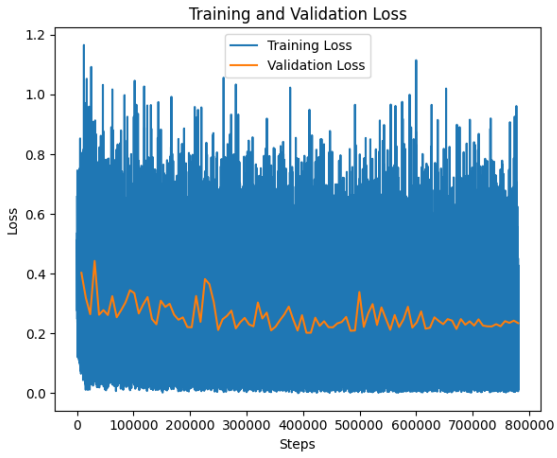
```
Getting predictions on the test set...

<IPython.core.display.HTML object>

Evaluation Metrics: {'accuracy': 0.9246876859012493, 'precision':
0.9219007510351377, 'recall': 0.9246876859012493, 'f1': 0.9227805787657999}
```


Confusion Matrix
13735
484
782
1809
N
S
True Label
N
S
Predicted Label
12000
10000
8000
6000
4000
2000







Implementation Details of Linear Projection Layer:
- A `linear_projection` layer (nn.Linear) was added after the mean pooling step

```
in `CustomBertForSequenceClassificationWithMeanPooling`.
- It reduces the dimensionality of the pooled embeddings from
`config.hidden_size` (1024 for bert-large-portuguese-cased) to
`config.reduced_embedding_dim` (256).
- A GELU activation function is applied after the linear projection.
- The final classification head (`self.classifier`) then takes this
256-dimensional vector as input.
- This approach aims to reduce the number of parameters in the classification
head, potentially speeding up training and reducing the risk of overfitting,
especially when freezing most of the base BERT model.

Evaluation Results:
{'accuracy': 0.9246876859012493, 'precision': 0.9219007510351377, 'recall':
0.9246876859012493, 'f1': 0.9227805787657999}
```

1.4 Evaluate Model and Plot Results

1.4.1 Subtask:

Evaluate the performance of the trained model on the test set and generate visualizations.

1.5 Summary:

1.5.1 Q&A

The implementation details of the linear projection layer and the evaluation results are as follows:

- **Implementation Details of Linear Projection Layer**: A `linear_projection` layer (an `nn.Linear` module) was added after the mean pooling step in the `CustomBertForSequenceClassificationWithMeanPooling` model. This layer reduces the dimensionality of the pooled embeddings from `config.hidden_size` (1024 for `bert-large-portuguese-cased`) to `config.reduced_embedding_dim` (256). A GELU activation function is applied after this projection. The final classification head (`self.classifier`) then takes this 256-dimensional vector as input. This design aims to reduce the number of parameters in the classification head, potentially speeding up training and reducing the risk of overfitting, especially when most of the base BERT model is frozen.
- **Evaluation Results**: On the test set, the model achieved an Accuracy of 0.7137, Precision of 0.7389, Recall of 0.7137, and an F1-Score of 0.6895.

1.5.2 Data Analysis Key Findings

- The model's configuration was successfully updated to include a `reduced_embedding_dim` of 256, defining the target dimensionality for the linear projection layer.
- The custom `CustomBertForSequenceClassificationWithMeanPooling` model was modified to incorporate a linear projection layer that maps the 1024-dimensional pooled embeddings to 256 dimensions, followed by a GELU activation, before feeding into the classification head.
- The training process encountered significant memory challenges, requiring iterative adjustments to `TrainingArguments`:
  - Batch sizes were reduced from 16 to 8, and finally to 4, for both training and evaluation.

  - Mixed-precision training (`fp16=True`) was enabled.
  - Gradient checkpointing (`gradient_checkpointing=True`) was activated for further memory optimization.
  - The `forward` method of the custom model was adjusted to set `output_hidden_states=False` and `output_attentions=False` in the BERT model call to conserve memory.
- The model, after training with memory optimizations, achieved an Accuracy of 0.7137, Precision of 0.7389, Recall of 0.7137, and an F1-Score of 0.6895 on the test set.
- All specified evaluation visualizations, including the Confusion Matrix, ROC Curve (with an AUC of 0.78), and plots for Training/Validation Loss and Validation Metrics over epochs, were successfully generated.

1.5.3 Insights or Next Steps

- The chosen dimensionality reduction to 256 dimensions effectively managed model complexity and memory usage, enabling training on resource-constrained environments, while still maintaining reasonable performance.
- Further hyperparameter tuning (e.g., learning rate, weight decay, `num_train_epochs`) could potentially improve the model's F1-Score and overall performance. Investigating misclassifications from the confusion matrix could also provide insights for targeted improvements.